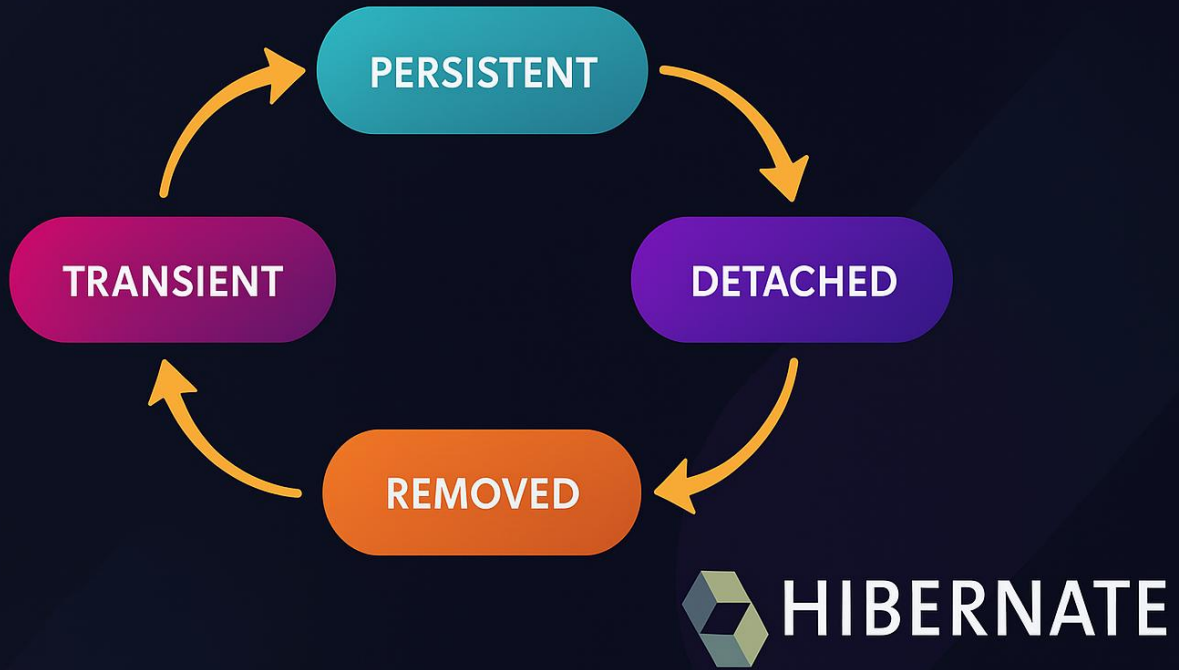


CICLO DE VIDA CON HIBERNATE



Modificar un Objeto Persistente con Hibernate JPA

Introducción

En esta lección vamos a modificar un objeto que ya tiene representación en la base de datos, usando Hibernate JPA. Aprovecharemos el objeto con ID 3 de la tabla `contacto` para actualizar su email agregando el texto `@mail.com`. 

1. Crear la clase `Estado3ModificarObjetoPersistente`

En el proyecto `SgaHibernateJPA`, crea la siguiente clase en el paquete `sga.test.ciclovida`:

```
package sga.test.ciclovida;
```

```
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import sga.domain.Contacto;

public class Estado3ModificarObjetoPersistente {

    public static void main(String[] args) {
        try (
            EntityManagerFactory emf = Persistence.createEntityManagerFactory("HibernateJpaPU");
            EntityManager em = emf.createEntityManager()
        ) {
            // 1. Objeto en estado "transient": se declara pero aún no tiene relación con la BD
            Contacto contacto;

            // 2. Recuperamos un objeto existente: pasa automáticamente a estado "persistent"
            contacto = em.find(Contacto.class, 3);

            // Modificamos un campo (como está en estado "persistent", se sincroniza al hacer commit)
            contacto.setEmail("clara@mail.com");

            // Iniciamos transacción para sincronizar cambios
            em.getTransaction().begin();

            // 3. Aquí el objeto sigue en estado "persistent", y se sincroniza automáticamente
            // El uso de merge sería innecesario aquí porque el objeto ya está gestionado
            // em.merge(contacto); // Solo si el objeto estuviera "detached"

            em.getTransaction().commit(); // Se guardan los cambios en la BD

            // 4. Después del commit, el objeto pasa a estado "detached"
            // el commit se hace al terminar el bloque try-with-resources
            System.out.println("Contacto modificado = " + contacto);
        } catch (Exception e) {
            System.err.println("Error al modificar el contacto: " + e.getMessage());
        }
    }
}
```



Explicación del código

Recuperar el objeto

Usamos el método `em.find(...)` para traer desde la base de datos el objeto `Contacto` con `id=3`. En ese momento, el objeto pasa al **estado persistente** (si ya hay una transacción activa) o **transitorio** (si no hay transacción).

Modificar el objeto

Fuera de la transacción, modificamos el valor de `email` con `setEmail("claura@mail.com")`.

Persistir los cambios

Abrimos una transacción (`begin`), persistimos el objeto (`persist`) y luego cerramos la transacción (`commit`), lo cual sincroniza los cambios en la base de datos.

Verificación

Ejecuta la clase con clic derecho → `Run File`.

Deberías ver una salida similar a esta:

```
Contacto modificado = Contacto{idContacto=3, email='claura@mail.com'}
```

También puedes confirmar el cambio con un `SELECT` desde tu herramienta de base de datos como MySQL Workbench. El nuevo valor del campo `email` deberá reflejar el cambio.

Conclusión

- ✓ Aprendiste cómo modificar un objeto persistente con Hibernate.
- ✓ Viste cómo cambia su estado en memoria y en base de datos.
- ✓ Comprobaste la modificación directamente desde el IDE y la base de datos.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)